

Edge AI UPI Behavioural Risk Intelligence System

Working Documentation

Generated on July 19, 2026

1. Project Summary

The Edge AI UPI Behavioural Risk Intelligence System is a fraud detection demo platform for digital payments. It combines behavioural analysis, machine learning, graph analytics, explainability, and a Streamlit dashboard to simulate how a modern fintech fraud monitoring system works.

The project includes:

- A FastAPI backend for fraud scoring and graph endpoints
- A Streamlit dashboard for visualization and testing
- Machine learning models for transaction risk scoring
- Graph-based fraud ring and suspicious node detection
- Explainability endpoints using SHAP

2. Verified Working Setup

The project was verified with the following runtime setup on July 19, 2026:

- Python 3.10
- FastAPI backend served from ``backend.main:app``
- Docker container exposing backend on ``127.0.0.1:8000``
- Streamlit dashboard running locally
- Local Python environment including ``pyarrow``

The Docker backend is expected to run on:

``http://127.0.0.1:8000``

The dashboard is expected to run on:

``http://localhost:8501``

3. Architecture Overview

The working flow is:

1. A user submits a transaction from the dashboard.
2. The FastAPI backend receives the request at ``/predict``.
3. Behavioural and rolling transaction features are computed.
4. The ensemble model produces a fraud probability.
5. The transaction is stored in ``transactions.csv``.
6. User-to-merchant edges are added to the fraud graph.
7. Dashboard pages query backend endpoints to render metrics, graphs, alerts, heatmaps, explanations, and GNN detection results.

Main components:

- ``backend/main.py``: active backend entrypoint
- ``dashboard/dashboard.py``: main Streamlit dashboard
- ``backend/models/ensemble_model.py``: fraud risk model loader
- ``backend/graph/graph_fraud_detector.py``: graph relationship store
- ``backend/advanced_ai/gnn_fraud_detector.py``: suspicious node heuristic

4. Important Fixes Applied

The following working fixes were identified and applied:

1. The README backend startup command was corrected from ``backend.api:app`` to ``backend.main:app``.
2. The backend Dockerfile was updated to run ``backend.main:app``.
3. The Docker image now installs packages from ``requirements.txt`` and also includes ``shap``.
4. ``pyarrow`` was added to ``requirements.txt`` to prevent local Streamlit rendering failures.
5. The Fraud Intelligence dashboard page now distinguishes backend request failures from dashboard rendering errors.

5. Local Installation

From the repository root:

```
pip install -r requirements.txt
```

Start the backend locally:

```
uvicorn backend.main:app --reload
```

Start the dashboard:

```
streamlit run dashboard/dashboard.py
```

6. Docker Backend Installation

Build the Docker image from the repository root:

```
docker build -f backend/Dockerfile -t edge-upi-backend .
```

Run the backend container:

```
docker run -d --name edge-upi-backend -p 8000:8000 edge-upi-backend
```

Useful Docker commands:

```
docker ps
```

```
docker logs -f edge-upi-backend
```

```
docker stop edge-upi-backend
```

```
docker start edge-upi-backend
```

7. Main Backend Endpoints

The active backend in `backend/main.py` exposes:

- `GET /` - backend status
- `POST /predict` - score a transaction
- `GET /transactions` - list all transactions
- `GET /heatmap` - heatmap data
- `GET /explain/{tx\_id}` - SHAP explainability
- `GET /fraud-graph` - graph edges
- `GET /fraud-rings` - suspicious rings
- `GET /temporal-patterns` - hourly fraud pattern view
- `GET /behavior/{tx\_id}` - behavioural score
- `GET /model-drift` - drift status
- `GET /gnn-fraud-detection` - suspicious nodes

Swagger UI:

`http://127.0.0.1:8000/docs`

8. Dashboard Pages

The dashboard includes:

- Fraud Intelligence
- Live Transaction Simulator
- Fraud Network Graph
- Fraud Rings
- Fraud Heatmap
- Explainability SHAP
- Fraud Alerts
- GNN Fraud Detection

These pages call the backend using `127.0.0.1:8000`.

9. How to Test the System

Use the Fraud Detection page in the dashboard and enter sample values such as:

- User ID: `user\_1001`
- Merchant: `merchant\_amazon`
- Amount: `2500`

High-risk style sample:

- User ID: `user\_1001`
- Merchant: `merchant\_suspicious\_01`
- Amount: `85000`

Expected flow:

1. Submit a transaction.
2. Check the returned transaction ID and risk score.
3. Open Fraud Network Graph to see user-merchant links.
4. Open Fraud Rings after multiple linked transactions.
5. Open Fraud Alerts to review risky transactions.
6. Open GNN Fraud Detection to detect suspicious nodes.

10. Troubleshooting

Connection Refused on Port 8000

If the dashboard shows:

```
`Failed to establish a new connection: [WinError 10061]`
```

Then the backend is not running. Start either:

- Local backend with ``uvicorn backend.main:app --reload``
- Docker backend with ``docker start edge-upi-backend``

Wrong Backend Entry Point

Do not use:

```
`uvicorn backend.api:app --reload`
```

Use:

```
`uvicorn backend.main:app --reload`
```

because the dashboard routes such as ``/gnn-fraud-detection`` are implemented in ``backend/main.py``.

No Module Named pyarrow

If Streamlit reports `No module named 'pyarrow'`, install project dependencies again:

```
pip install -r requirements.txt
```

Model Version Warnings

The backend may print scikit-learn model version mismatch warnings while loading pickled models. These warnings do not currently block startup, but model serialization and runtime versions should be aligned for long-term stability.

11. Current Working Status

As of July 19, 2026:

- The Docker backend responds on `GET /gnn-fraud-detection`
- The backend serves Swagger at `/docs`
- The local Python environment has `pyarrow`
- The dashboard can connect to the backend on port `8000`

12. Repo Files Most Relevant to Operation

- `README.md`
- `requirements.txt`
- `backend/Dockerfile`
- `backend/main.py`
- `dashboard/dashboard.py`
- `dashboard/pages/0\_Fraud\_Intelligence.py`

13. Conclusion

The project is now documented around the backend and dashboard path that was actually verified to work. For the most reliable startup path, run the backend from ``backend.main:app`` and keep the Docker backend bound to port ``8000`` before launching the Streamlit dashboard.